

Cortex-M7: ETMv4 Application Note



This document and all documents accompanying it are copyrighted by iSYSTEM and all rights are reserved. Duplication of these documents is allowed for personal use. For every other case, written consent from iSYSTEM is required.

Copyright © iSYSTEM, AG.

All rights reserved.

Table of Contents

1.	Introduction	2
1.1	Embedded Trace Macrocell (ETM).....	2
1.2	Data Watchpoint and Trace (DWT).....	3
2.	Record Code Execution	4
3.	Productivity tips	19
1.3	Default Trigger	19
1.4	Configure Profiler	19
1.5	Search for variables via the Symbol Browser.....	20
1.6	Discover the end of the selected function via the Symbol Browser	20
1.7	Available DWT Comparators	20
1.8	Continuously Record Code Execution “from – until”	21
4.	Technical support	24
1.9	More resources	24
1.10	Contact	24

1. Introduction

This document describes how to record only a section of code execution on Cortex-M7 architecture featuring ETMv4.

Screenshots were taken from a sample application running on ST STM32H755ZI device, which features a high-performance 32-bit ARM Cortex-M7 in conjunction with 32-bit Cortex-M4 core.

ETMv4 is versatile and much more complicated in comparison to ETMv3 found on ARM Cortex-M3/M4. This document explains different trace configurations and possibilities.

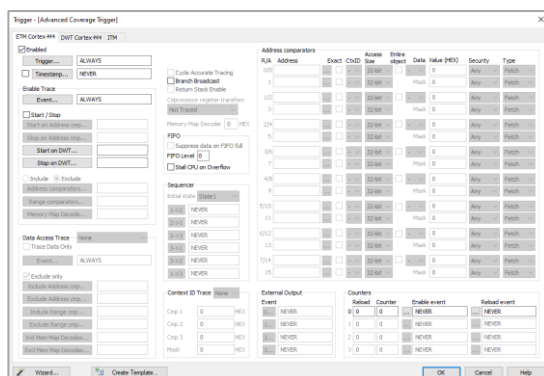
For more information refer to ARM i.e., *ARM CoreSight ETM-M7 Technical Reference Manual* and/or MCU Vendor documentation.

1.1 Embedded Trace Macrocell (ETM)

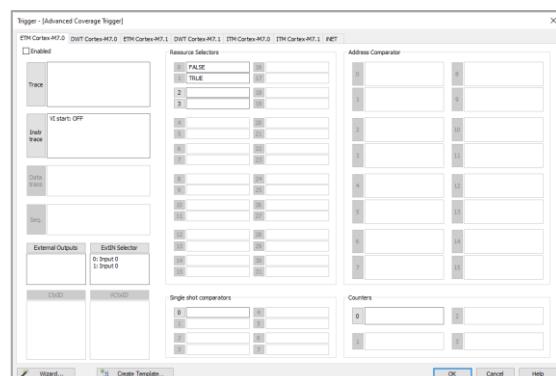
ETM is supported on ARM Cortex-M, Cortex-A and Cortex-R. Trace module comes in a couple of versions. ARM Cortex-M ETM variants usually don't feature data trace and own comparators while Cortex-A/R ETM is usually fully featured trace module.

Depending on the processor implementation ARMv8 systems can have ETMv4 or ETMv3. For example, ARMv8-A systems have ETMv4 only and ARMv8-M might have ETMv4 or ETMv3.

ARM Cortex-M device features either ETMv3 or ETMv4 trace module. Based on the selected device winIDEA provides the according ETM Trigger Configuration dialog.



ETMv3 Trigger dialog



ETMv4 Trigger dialog

1.1.1. ETMv3

ETMv3 supports instruction and data trace. ETMv3 is present on e.g. ARM Cortex-M3/M4 and still used widely today for low-power devices (ARM Cortex-M/R). Trace formatters, e.g., Trace Port Interface Unit (TPIU), Embedded Trace Buffer (ETB) etc. insert the trigger data pattern into the stream on conditions that are configured into different trace sources (ETM, DWT...).

1.1.2. ETMv4

ETMv4 is present on e.g., ARM Cortex-M7/M33 and it inserts a trigger directly into the stream. There is no need for Trace formatters (TPIU, ETB...) to do so.

1.2 Data Watchpoint and Trace (DWT)

DWT is supported on ARM Cortex-M. It provides up to 4 memory access Comparators that can be used in combination with ITM and/or ETM for a low bandwidth focused data trace.

1.1.3. DWT Comparators

Each DWT Comparator can be configured for a specific address. Directly there is no address range implemented but all Comparators have an LSB (Least Significant Bit) masking register which implements an ignore mask. The size of the ignore mask is 0-31 bits, applied to (indirectly) address range matching. The range can be specified via winIDEA option *Ignore LSB bits* drop-down menu.

Comparator 0 can additionally count and compare against the Clock Cycle Counter (CYCCNT).

Comparator 1 can be additionally configured for data value comparison (Data Comparator) for a given address. This works only for action CMPMATCH (Compare MATCH). Comparator 1 cannot be used neither for qualifying nor for recording data values in this configuration. It can be configured to use two addresses (LSB masking works too), and data value (uses Comparator 3 internal resources which become unavailable for the user and disabled in the GUI). If 2nd address is used, Comparator 2 internal resources are used and also become unavailable for the user).

A full DWT module contains four comparators that you can configure as:

- Hardware watchpoint,
- ETM trigger,
- PC sampler event trigger and PC sampler together with Data access and its value,
- Data address sampler event trigger.

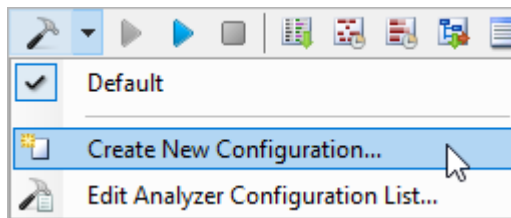
2. Record Code Execution

Follow the steps below and configure a new trigger in the winIDEA Analyzer:

1. [Create a new Trigger Configuration.](#)
2. [Select On Trigger.](#)
3. [Configure ETM for the M7 core.](#)
4. [Configure DWT for the M7 core.](#)
5. [Configure ITM for the M7 core.](#)

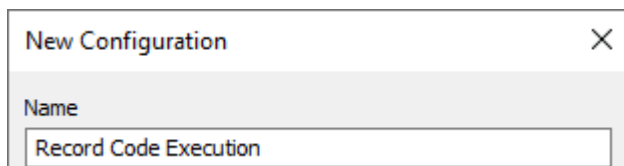
1 Create a new Trigger Configuration.

1. Open View menu / Analyzer / Analyzer Configuration button
2. Select *Create New Configuration.*

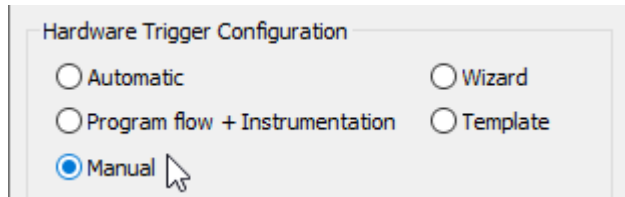


➤ See productivity tip [Default Trigger.](#)

3. Give it a meaningful name.



4. Select *Manual Hardware Trigger Configuration* and confirm.

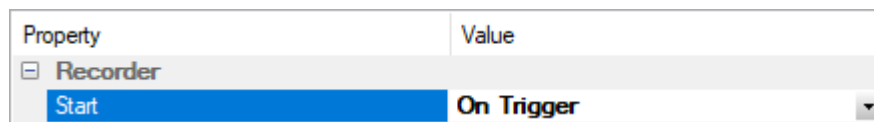


Trigger – [Record Code Execution] configuration window will open on the *Recorder* page.

2

Select *On Trigger*.

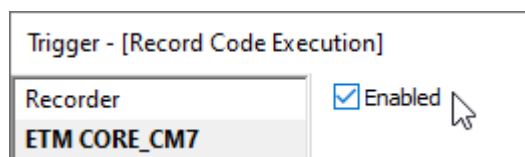
Make sure the sampling start point is set to *On Trigger* on the *Recorder* page.



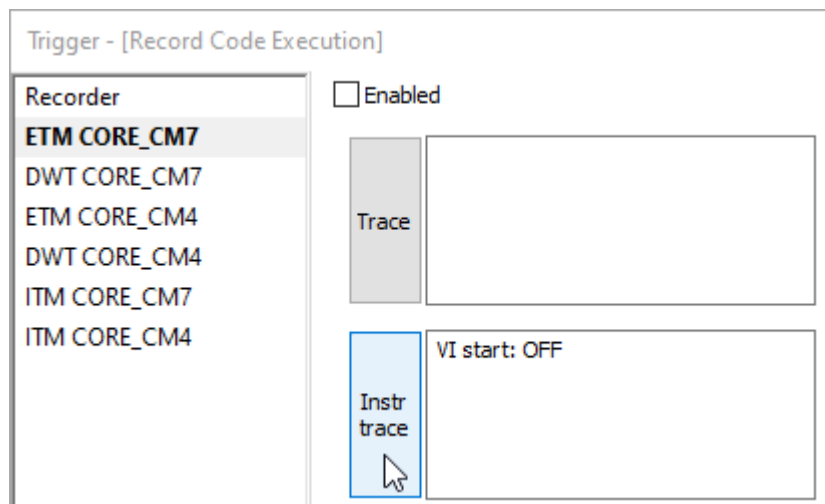
3

Configure ETM for the M7 core.

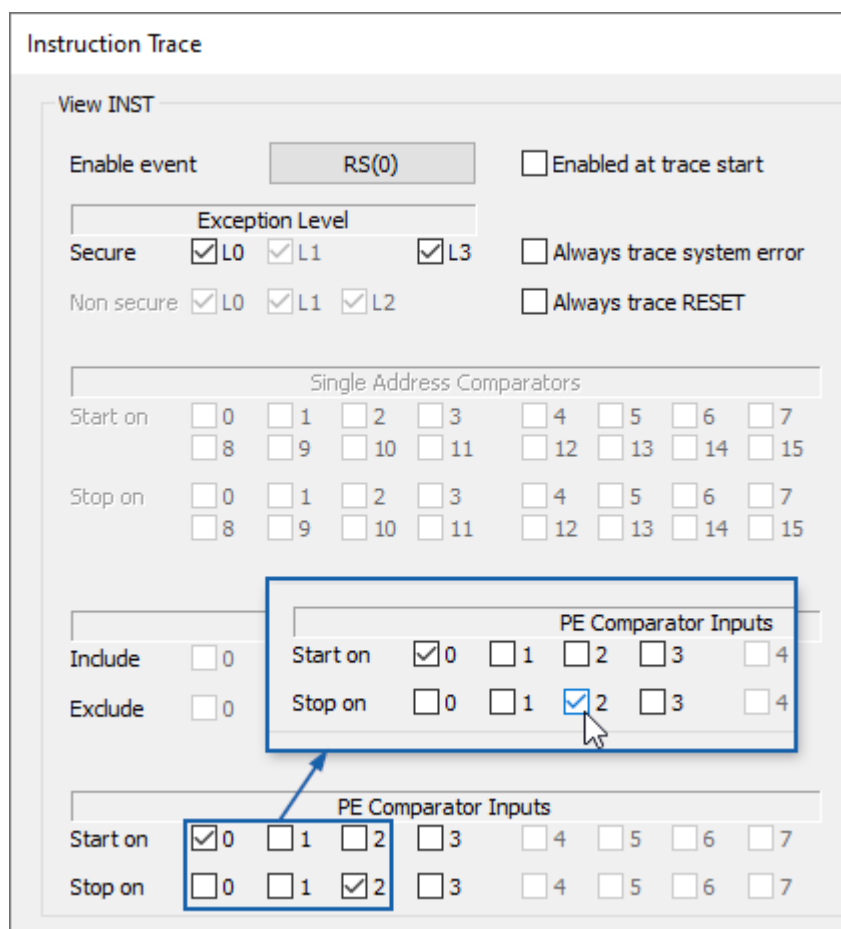
1. Go on the *ETM CORE_CM7* page and check the *Enabled* checkbox.



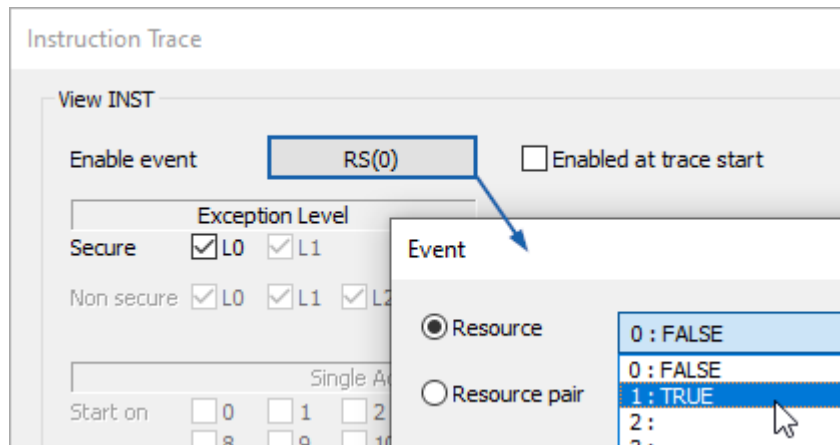
2. Press *Instr trace* to open the *Instruction Trace* dialog.



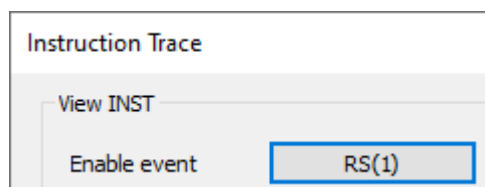
3. Set the *PE Comparator Inputs* to **start trace on PE0**, which connects to the DWT Comparator 0 CMPMATCH output and to **stop on PE2**, which connects to the DWT Comparator 2 CMPMATCH output. This will signal to Instruction Trace a start and an end activation event.



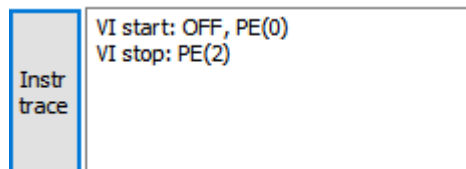
4. Press *RS (0)* and select *1: TRUE* in the *Resource* drop-down menu.



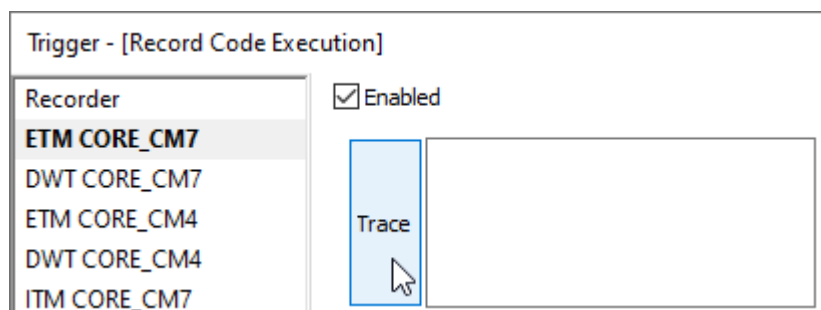
View INST section is now set.



5. Press OK. The *Instr trace* section is now set.



6. To get the trigger indication press *Trace* from *ETM CORE_CM7* page.



7. Check *Enable ATB trigger*.

Trace

Core select

☐ Trace ContextID ☐ Trace Virtual ContextID

☐ Enable Cycle counter Threshold

☐ Auto global timestamp On event

Synchronization period

☒ Enable ATB trigger

8. Press OK. *Trace* is now configured.

Trace

Opt:ATB

9. To get the trigger position you need to setup *Resource Selectors* section.
- Press to add a new *Resource Selector* which is connected to the PE0.
 - Select *PE Comparator Inputs* from the *Group* drop-down.
 - Check *PE Com In 0*.

Resource Selectors

0	FALSE	16	
1	TRUE	17	
2		18	
3		19	

Address Compar

0	
1	

Resource Selector

Group

☐ Invert output ☐ Invert PAIR function output

☒ PE Comp In 0

- d. Press OK. *Resource Selectors* section is now set.

Resource Selectors	
0	FALSE
1	TRUE
2	PE(0)
3	

- See productivity tip [Record Code Execution repeatedly from – until](#) and set additional *Resource Selector*.

10. Press *External Outputs* and add one.
- Check option *Generate instruction trace event*.
 - Press *RS(0)* in the section 0.
 - Select 2: *PE(0)* from the *Resource* drop-down.

External Outputs

ExtIN Selector

0: Input 0

External Output

0

Event RS(0)

☒ Generate instruction trace event

☐ Generate resource pair event

1

Event

☐ Resource

☐ Resource pair

0 : FALSE

0 : FALSE

1 : TRUE

2 : PE(0)

3 :

- d. Press OK. *External Outputs* are now set.

External Outputs	
0:	I, RS(2)

- See productivity tip [Record Code Execution repeatedly from – until](#) and set additional *External Output*.

4

Configure DWT for the M7 core.

DWT Trace will be configured to record writes to a 16-bit variable named `interfcoun`.

Note that not always ETM transmits immediately a belonging message for every 8-bit or 16-bit variable data write, but combines this into two consecutive messages being transmitted latter in conjunction with a 2nd data write event.

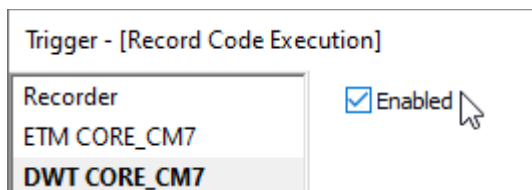
The example below issues outputs towards the ETM part if the function `Systick_Handler` Interrupt is entered or exited. It will also record write accesses to the variable `interfcoun` within the Interrupt function. In our example this variable is written only within this part of the code:

```
volatile unsigned short interfcount;

void InterruptRoutine()
{
    // counting 1 ms Interrupts before incrementing variable depending on
    // value of iWAITVALUE (char)

    volatile static char iWait;
    if ((iWait++) > iWAITVALUE)
    {
        iInterruptCounter7++;
        iWait = 0;
    }
    interfcount++;
}
```

1. Go to the *DWT CORE_CM7* page and check the *DWT Enabled* checkbox.



2. Enable Comparator 0:
 - a. Select *Instruc. Execution* from the drop-down menu.
 - b. Set the Address to `Systick_Handler`.

☒ Comparator 0

Instruct. Execution ▼

Address ... Ignore LSB bits ▼

Cycle counter HEX

Action generate External (CMPMATCH) output ▼

➤ See productivity tip [Search for variables via the Symbol Browser](#).

3. Enable Comparator 1:

- a. Select *Write Access* from the drop-down menu.
- b. Set the *Address 1* to *interfcount*.
- c. Select *sample data value* from the *Action* drop-down menu.

☒ Comparator 1

Write Access ▼

☒ Address 1 ... Ignore LSB bits ▼

☐ Address 2 ... Ignore LSB bits ▼

☐ Data HEX Data Size 8 bit ▼

Action sample data value ▼

4. Enable Comparator 2:
 - a. Select *Instruct. Execution* from the drop-down menu.
 - b. Set the Address to `Systick_Handler_EXIT`.

Comparator 2

Instruct. Execution

Address: Systick_Handler_EXIT ... Ignore LSB bits: 0

Action: generate External (CMPMATCH) output

- See productivity tip [Discover the end of the selected function via the Symbol Browser](#).
- See productivity tip [Configure unused DWT Comparators](#).

5

Configure ITM for the M7 core.

Note: ITM in the *ITM* page must be checked too for the DWT to work, regardless if ITM is used for its primary use or not.

1. Go to the *ITM CORE_CM7* page and check the *ITM Enabled* checkbox.

Trigger - [Record Code Execution]

Recorder

- ETM CORE_CM7
- DWT CORE_CM7
- ETM CORE_CM4
- DWT CORE_CM4
- ITM CORE_CM7**

☒ Enabled

Timestamps

☐ Local

☐ Global

2. Set *Record Stimuli* to *All* to see variable value.

Record Stimuli: All

0 - 7 ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ Priv ☐

8 - 15 ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ Priv ☐

1.1.4. Results of the recording

After you start the application after `main()` you will get the below result. Only the `SysTick_Handler` code section and **write accesses** to a variable `interfcnt` (because of DWT works independent) will get recorded.

Note that the DWT/TPIU can pack and transmit messages for 8- or 16-bit variables, which occur at different point in time, in a single block of messages. This could be misleading for the user but it's the way how ETM/DWT works to get maximal efficiency in terms of data trace throughput. To improve the settings an extensive understanding and knowledge of ARM's documentation is required.

1.2.1.1 Case A

Activate trace first, then start the execution from reset on with the [trigger notification setup](#). Results of the recording are displayed automatically.

Note the two `startup.c` writes to a variable `interfcoun`, but trigger works correctly.

Markers in the below recordings are:



– Trigger



– ETM4 event



– Next event/interrupt

sample_gcc_ic5k7_21x.trd* [Record Code Execution from ... til]						
Trace						
	Number	Address	Data	Content	Time	Core
T	0.0	0000'1001	00000001	Trigger HWID=1001	0 ns	CORE_CM7
	7.0	2000'0016	00000000	interfcoun dwTestWord7 [ITM] Data Write	1.019600 ms	CORE_CM7
1	18.0	0000'0000	00000001	[ETM4] Event	1.656 us	CORE_CM7
	18.1	2000'0016	00000001	interfcoun [ITM] Data Write	1.019728 ms	CORE_CM7
	22.0	0800'08D4	0000B580	{ SysTick_Handler	1.019792 ms	CORE_CM7
	22.1	0800'08D8	FF3CF7FF	InterruptRoutine();	1.019801 ms	CORE_CM7
	22.2	0800'0754	0000B480	{ InterruptRoutine	1.019805 ms	CORE_CM7
	22.3	0800'0758	00004B0E	if ((iWait++) > iWAITVALUE)	1.019815 ms	CORE_CM7
	22.4	0800'077E	00004B08	interfcoun++;	1.019865 ms	CORE_CM7
	22.5	0800'078A	0000BF00	}	1.019893 ms	CORE_CM7
	22.6	0800'0792	00004770	InterruptRoutine_EXIT	1.019907 ms	CORE_CM7
	22.7	0800'08DC	0000BF00	}	1.019912 ms	CORE_CM7
	22.8	0800'08DE	0000BD80	SysTick_Handler_EXIT	1.529812 ms	CORE_CM7
C	26.0	0000'0000	00000001	[ETM4] Event	2.039712 ms	CORE_CM7
	26.1	0800'08D4	0000B580	{ SysTick_Handler	2.039714 ms	CORE_CM7
	26.2	0800'08D8	FF3CF7FF	InterruptRoutine();	2.039719 ms	CORE_CM7
	26.3	0800'0754	0000B480	{ InterruptRoutine	2.039721 ms	CORE_CM7
	26.4	0800'0758	00004B0E	if ((iWait++) > iWAITVALUE)	2.039726 ms	CORE_CM7
	26.5	0800'077E	00004B08	interfcoun++;	2.039752 ms	CORE_CM7
	26.6	0800'078A	0000BF00	}	2.039766 ms	CORE_CM7
	26.7	0800'0792	00004770	InterruptRoutine_EXIT	2.039773 ms	CORE_CM7
	26.8	0800'08DC	0000BF00	}	2.039776 ms	CORE_CM7
	26.9	0800'08DE	0000BD80	SysTick_Handler_EXIT	2.549800 ms	CORE_CM7
	31.0	2000'0016	00000002	interfcoun [ITM] Data Write	3.059824 ms	CORE_CM7
	31.1	0000'0000	00000001	[ETM4] Event	3.059888 ms	CORE_CM7
	31.2	0800'08D4	0000B580	{ SysTick_Handler	3.059952 ms	CORE_CM7
	31.3	0800'08D8	FF3CF7FF	InterruptRoutine();	3.059956 ms	CORE_CM7

1.2.1.2 Case B

Case B is similar to [Case A](#) but execution starts from `main()`. This example shows data buffer writes.

Markers in the below recordings are:



– Trigger



– ETM4 event



– Next event/interrupt

sample_gcc_ic5k7_21x.trd* [Record Code Execution from ... till]						
Trace						
	Number	Address	Data	Content	Time	Core
T	0.0	0800'0900	00004B37	iCounter7, testCounter7, iInterruptCounter7 = 0;	0 ns	CORE_CM7
	0.1	0000'1001	00000001	Trigger HWID=1001	127.633664 ms	CORE_CM7
	14.0	0000'0000	00000001	[ETM4] Event	128.653336 ms	CORE_CM7
	18.0	0800'08D4	0000B580	{ SysTick_Handler	128.653584 ms	CORE_CM7
	18.1	0800'08D8	FF3CF7FF	InterruptRoutine();	128.653593 ms	CORE_CM7
	18.2	0800'0754	0000B480	{ InterruptRoutine	128.653598 ms	CORE_CM7
	18.3	0800'0758	00004B0E	if ((iWait++) > iWAITVALUE)	128.653608 ms	CORE_CM7
	18.4	0800'077E	00004B08	interfcount++;	128.653662 ms	CORE_CM7
	18.5	0800'078A	0000BF00	}	128.653692 ms	CORE_CM7
	18.6	0800'0792	00004770	InterruptRoutine_EXIT_	128.653707 ms	CORE_CM7
	18.7	0800'08DC	0000BF00	}	128.653712 ms	CORE_CM7
	18.8	0800'08DE	0000BD80	SysTick_Handler_EXIT_	129.163632 ms	CORE_CM7
	18.9	2000'0016	00000001	interfcount [ITM] Data Write	129.673552 ms	CORE_CM7
	22.0	2000'0016	00000002	interfcount [ITM] Data Write	129.673680 ms	CORE_CM7
	22.1	0000'0000	00000001	[ETM4] Event	129.673736 ms	CORE_CM7
	22.2	0800'08D4	0000B580	{ SysTick_Handler	129.711499 ms	CORE_CM7
	22.3	0800'08D8	FF3CF7FF	InterruptRoutine();	129.787025 ms	CORE_CM7
	22.4	0800'0754	0000B480	{ InterruptRoutine	129.824789 ms	CORE_CM7
	22.5	0800'0758	00004B0E	if ((iWait++) > iWAITVALUE)	129.900315 ms	CORE_CM7
	22.6	0800'077E	00004B08	interfcount++;	130.315711 ms	CORE_CM7
	22.7	0800'078A	0000BF00	}	130.542290 ms	CORE_CM7
	22.8	0800'0792	00004770	InterruptRoutine_EXIT_	130.655580 ms	CORE_CM7
	22.9	0800'08DC	0000BF00	}	130.693344 ms	CORE_CM7
	22.10	0800'08DE	0000BD80	SysTick_Handler_EXIT_	130.693532 ms	CORE_CM7
	26.0	0000'0000	00000001	[ETM4] Event	130.693720 ms	CORE_CM7
	26.1	0800'08D4	0000B580	{ SysTick_Handler	130.693784 ms	CORE_CM7
	26.2	0800'08D8	FF3CF7FF	InterruptRoutine();	130.693894 ms	CORE_CM7
	26.3	0800'0754	0000B480	{ InterruptRoutine	130.693949 ms	CORE_CM7
	26.4	0800'0758	00004B0E	if ((iWait++) > iWAITVALUE)	130.694059 ms	CORE_CM7
	26.5	0800'077E	00004B08	interfcount++;	130.694665 ms	CORE_CM7
	26.6	0800'078A	0000BF00	}	130.694995 ms	CORE_CM7
	26.7	0800'0792	00004770	InterruptRoutine_EXIT_	130.695160 ms	CORE_CM7
	26.8	0800'08DC	0000BF00	}	130.695216 ms	CORE_CM7
	26.9	0800'08DE	0000BD80	SysTick_Handler_EXIT_	130.695432 ms	CORE_CM7
	30.0	2000'0016	00000003	interfcount [ITM] Data Write	130.695648 ms	CORE_CM7
	30.1	0000'0000	00000001	[ETM4] Event	131.715216 ms	CORE_CM7
	30.2	2000'0016	00000004	interfcount [ITM] Data Write	132.733936 ms	CORE_CM7
	34.0	0800'08D4	0000B580	{ SysTick_Handler	131.715280 ms	CORE_CM7
	34.1	0800'08D8	FF3CF7FF	InterruptRoutine();	131.715284 ms	CORE_CM7
	34.2	0800'0754	0000B480	{ InterruptRoutine	131.715287 ms	CORE_CM7
	34.3	0800'0758	00004B0E	if ((iWait++) > iWAITVALUE)	131.715292 ms	CORE_CM7
	34.4	0800'077E	00004B08	interfcount++;	131.715319 ms	CORE_CM7
	34.5	0800'078A	0000BF00	}	131.715334 ms	CORE_CM7
	34.6	0800'0792	00004770	InterruptRoutine_EXIT_	131.715341 ms	CORE_CM7
	34.7	0800'08DC	0000BF00	}	131.715344 ms	CORE_CM7
	34.8	0800'08DE	0000BD80	SysTick_Handler_EXIT_	132.224640 ms	CORE_CM7
	38.0	0000'0000	00000001	[ETM4] Event	132.734056 ms	CORE_CM7
	38.1	0800'08D4	0000B580	{ SysTick_Handler	132.734058 ms	CORE_CM7
	38.2	0800'08D8	FF3CF7FF	InterruptRoutine();	132.734063 ms	CORE_CM7

1.2.1.3 Case C

This example shows trace activation during runtime program execution.

Internal Trace registers and their internal time stamps consumption yield results displayed below. Ignore the lines before line 18.0 within the next result.

Markers in the below recordings are:



– Trigger



– ETM4 event



– Next event/interrupt

sample_gcc_ic5k7_21x.trd* [Record Code Execution from ... til]						
Trace						
	Number	Address	Data	Content	Time	Core
T	0.0	0000'1001	00000001	Trigger HWID=1001	9.179000 ms	CORE_CM7
	6.0	2000'0016	00006FC5	interfcouint [ITM] Data Write	5.100152 ms	CORE_CM7
	10.0	2000'0016	00006FC6	interfcouint [ITM] Data Write	5.100208 ms	CORE_CM7
	10.1	2000'0016	00006FC7	interfcouint [ITM] Data Write	5.100336 ms	CORE_CM7
	10.2	2000'0016	00006FC8	interfcouint [ITM] Data Write	5.100400 ms	CORE_CM7
	10.3	2000'0016	00006FC9	interfcouint [ITM] Data Write	9.178408 ms	CORE_CM7
	10.4	2000'0016	00006FCA	interfcouint [ITM] Data Write	9.178536 ms	CORE_CM7
	13.0	2000'0016	00006FCB	interfcouint [ITM] Data Write	9.178600 ms	CORE_CM7
	13.1	2000'0016	00006FCC	interfcouint [ITM] Data Write	9.178720 ms	CORE_CM7
	13.2	2000'0016	00006FCD	interfcouint [ITM] Data Write	9.178784 ms	CORE_CM7
	13.3	2000'0016	00006FCE	interfcouint [ITM] Data Write	9.180464 ms	CORE_CM7
1	18.0	0000'0000	00000001	[ETM4] Event	9.180344 ms	CORE_CM7
	18.1	0800'08D4	0000B580	{ SysTick_Handler	10.200192 ms	CORE_CM7
	18.2	0800'08D8	FF3CF7FF	InterruptRoutine();	10.200201 ms	CORE_CM7
	18.3	0800'0754	0000B480	{ InterruptRoutine	10.200206 ms	CORE_CM7
	18.4	0800'0758	00004B0E	if ((iWait++) > iWAITVALUE)	10.200216 ms	CORE_CM7
	18.5	0800'077E	00004B08	interfcouint++;	10.200270 ms	CORE_CM7
	18.6	0800'078A	0000BF00	}	10.200300 ms	CORE_CM7
	18.7	0800'0792	00004770	InterruptRoutine_EXIT_	10.200315 ms	CORE_CM7
	18.8	2000'0016	00006FCF	interfcouint [ITM] Data Write	11.218800 ms	CORE_CM7
	22.0	0800'08DC	0000BF00	}	10.200320 ms	CORE_CM7
	22.1	0800'08DE	0000BD80	SysTick_Handler_EXIT_	10.709560 ms	CORE_CM7
	26.0	2000'0016	00006FDD	interfcouint [ITM] Data Write	11.218928 ms	CORE_CM7
C	26.1	0000'0000	00000001	[ETM4] Event	11.218992 ms	CORE_CM7
	26.2	0800'08D4	0000B580	{ SysTick_Handler	11.218994 ms	CORE_CM7
	26.3	0800'08D8	FF3CF7FF	InterruptRoutine();	11.218998 ms	CORE_CM7
	26.4	0800'0754	0000B480	{ InterruptRoutine	11.219000 ms	CORE_CM7
	26.5	0800'0758	00004B0E	if ((iWait++) > iWAITVALUE)	11.219004 ms	CORE_CM7
	26.6	0800'077E	00004B08	interfcouint++;	11.219027 ms	CORE_CM7
	26.7	0800'078A	0000BF00	}	11.219039 ms	CORE_CM7
	26.8	0800'0792	00004770	InterruptRoutine_EXIT_	11.219045 ms	CORE_CM7
	26.9	0800'08DC	0000BF00	}	11.219048 ms	CORE_CM7
	26.10	0800'08DE	0000BD80	SysTick_Handler_EXIT_	11.729000 ms	CORE_CM7
	30.0	0000'0000	00000001	[ETM4] Event	12.238952 ms	CORE_CM7
	30.1	0800'08D4	0000B580	{ SysTick_Handler	12.239008 ms	CORE_CM7
	30.2	0800'08D8	FF3CF7FF	InterruptRoutine();	12.239012 ms	CORE_CM7

Without the [trigger notification setup](#) in the previous steps trace state remains in the WAITING state

WAITING

and you must manually stop  the recording to display the results.

sample_gcc_ic5k7_21x.trd* [Record Code Execution from ... til] X						
Trace						
	Number	Address	Data	Content	Time	Core
C	-16378.0	2000'0016	0000DBC	interfcoun [ITM] Data Write	4.079280 ms	CORE_CM7
	-16375.0	2000'0016	0000DBD0	interfcoun [ITM] Data Write	4.079400 ms	CORE_CM7
	-16375.1	2000'0016	0000DBD1	interfcoun [ITM] Data Write	4.079464 ms	CORE_CM7
	-16375.2	2000'0016	0000DBD2	interfcoun [ITM] Data Write	4.079592 ms	CORE_CM7
	-16375.3	2000'0016	0000DBD3	interfcoun [ITM] Data Write	4.079648 ms	CORE_CM7
	-16375.4	2000'0016	0000DBD4	interfcoun [ITM] Data Write	9.177272 ms	CORE_CM7
	-16371.0	2000'0016	0000DBD5	interfcoun [ITM] Data Write	9.177400 ms	CORE_CM7
	-16371.1	2000'0016	0000DBD6	interfcoun [ITM] Data Write	9.177456 ms	CORE_CM7
	-16371.2	2000'0016	0000DBD7	interfcoun [ITM] Data Write	9.177584 ms	CORE_CM7
	-16371.3	2000'0016	0000DBD8	interfcoun [ITM] Data Write	9.179264 ms	CORE_CM7
	-16363.0	0800'08D4	0000B580	{ SysTick_Handler	10.198984 ms	CORE_CM7
	-16363.1	0800'08D8	FF3CF7FF	InterruptRoutine();	10.198993 ms	CORE_CM7
	-16363.2	0800'0754	0000B480	{ InterruptRoutine	10.198997 ms	CORE_CM7
	-16363.3	0800'0758	00004B0E	if ((iWait++) > iWAITVALUE)	10.199007 ms	CORE_CM7
	-16363.4	0800'077E	00004B08	interfcoun++;	10.199057 ms	CORE_CM7
	-16363.5	0800'078A	0000BF00	}	10.199085 ms	CORE_CM7
	-16363.6	0800'0792	00004770	InterruptRoutine_EXIT	10.199099 ms	CORE_CM7
	-16363.7	2000'0016	0000DBD9	interfcoun [ITM] Data Write	10.199296 ms	CORE_CM7
	-16359.0	0800'08DC	0000BF00	}	10.199104 ms	CORE_CM7
	-16359.1	0800'08DE	0000BD80	SysTick_Handler_EXIT	10.199200 ms	CORE_CM7
	-16359.2	2000'0016	0000DBDA	interfcoun [ITM] Data Write	12.237232 ms	CORE_CM7
	-16359.3	0800'08D4	0000B580	{ SysTick_Handler	12.237360 ms	CORE_CM7
	-16359.4	0800'08D8	FF3CF7FF	InterruptRoutine();	12.237364 ms	CORE_CM7
	-16359.5	0800'0754	0000B480	{ InterruptRoutine	12.237366 ms	CORE_CM7
	-16359.6	0800'0758	00004B0E	if ((iWait++) > iWAITVALUE)	12.237370 ms	CORE_CM7
	-16359.7	0800'077E	00004B08	interfcoun++;	12.237394 ms	CORE_CM7
	-16359.8	0800'078A	0000BF00	}	12.237407 ms	CORE_CM7
	-16359.9	0800'0792	00004770	InterruptRoutine_EXIT	12.237413 ms	CORE_CM7
	-16354.0	0800'08DC	0000BF00	}	12.237416 ms	CORE_CM7
	-16354.1	0800'08DE	0000BD80	SysTick_Handler_EXIT	12.237544 ms	CORE_CM7
	-16354.2	0800'08D4	0000B580	{ SysTick_Handler	12.237672 ms	CORE_CM7
	-16354.3	0800'08D8	FF3CF7FF	InterruptRoutine();	12.237676 ms	CORE_CM7
	-16354.4	0800'0754	0000B480	{ InterruptRoutine	12.237678 ms	CORE_CM7
	-16354.5	0800'0758	00004B0E	if ((iWait++) > iWAITVALUE)	12.237682 ms	CORE_CM7
	-16354.6	0800'077E	00004B08	interfcoun++;	12.237706 ms	CORE_CM7
	-16354.7	0800'078A	0000BF00	}	12.237719 ms	CORE_CM7
	-16354.8	0800'0792	00004770	InterruptRoutine_EXIT	12.237725 ms	CORE_CM7
	-16354.9	0800'08DC	0000BF00	}	12.237728 ms	CORE_CM7
	-16354.10	0800'08DE	0000BD80	SysTick_Handler_EXIT	12.238632 ms	CORE_CM7
	-16350.0	2000'0016	0000DBDB	interfcoun [ITM] Data Write	12.239536 ms	CORE_CM7
	-16350.1	0800'08D4	0000B580	{ SysTick_Handler	12.239720 ms	CORE_CM7
	-16350.2	0800'08D8	FF3CF7FF	InterruptRoutine();	12.318107 ms	CORE_CM7

1.2.1.4 Case D

This recording was configured according to [Record Code Execution repeatedly from – until](#) and trace activation during runtime.

Ignore lines 0.0 until 21.0 and the  in this case.

 – First recording

, ,  – Event notifications

sample_gcc_ic5k7_21x.trd* [Record Code Execution from ... til] -> X							
Trace							
SysTick_Handler							
	Number	Address	Data	Content	Time	Core	
	0.0	0000*1001	00000001	Trigger HWID=1001	7.140968 ms	CORE_CM7	
	6.0	2000*0016	00006980	interfcoun	0 ns	CORE_CM7	
				{ITM} Data Write			
	9.0	2000*0016	00006981	interfcoun	120 ns	CORE_CM7	
				{ITM} Data Write			
	9.1	2000*0016	00006982	interfcoun	184 ns	CORE_CM7	
				{ITM} Data Write			
	9.2	2000*0016	00006983	interfcoun	304 ns	CORE_CM7	
				{ITM} Data Write			
	9.3	2000*0016	00006984	interfcoun	368 ns	CORE_CM7	
				{ITM} Data Write			
	9.4	2000*0016	00006985	interfcoun	5.101768 ms	CORE_CM7	
				{ITM} Data Write			
	15.0	2000*0016	00006986	interfcoun	5.101888 ms	CORE_CM7	
				{ITM} Data Write			
	15.1	2000*0016	00006987	interfcoun	5.101952 ms	CORE_CM7	
				{ITM} Data Write			
	15.2	2000*0016	00006988	interfcoun	5.102080 ms	CORE_CM7	
				{ITM} Data Write			
	15.3	2000*0016	00006989	interfcoun	6.120616 ms	CORE_CM7	
				{ITM} Data Write			
	15.4	2000*0016	0000698A	interfcoun	6.122424 ms	CORE_CM7	
				{ITM} Data Write			
	21.0	2000*0016	0000698B	interfcoun	6.122544 ms	CORE_CM7	
				{ITM} Data Write			
	25.0	0800*08D4	0000B580	{	7.140880 ms	CORE_CM7	
				SysTick_Handler			
	25.1	0800*08D8	FF3CF7FF	InterruptRoutine();	7.140888 ms	CORE_CM7	
	25.2	0800*0754	0000B480	{	7.140892 ms	CORE_CM7	
				InterruptRoutine			
	25.3	0800*0758	00004B0E	if ((iWait++) > iWAITVALUE)	7.140900 ms	CORE_CM7	
	25.4	0000*0000	00000001	{ETM4} Event	7.140944 ms	CORE_CM7	
	25.5	0800*077E	00004B08	interfcoun++;	7.140955 ms	CORE_CM7	
	25.6	0800*078A	0000BF00	}	7.141025 ms	CORE_CM7	
	25.7	0800*0792	00004770	InterruptRoutine_EXIT	7.141060 ms	CORE_CM7	
	25.8	0800*08DC	0000BF00	}	7.141072 ms	CORE_CM7	
	25.9	0800*08DE	0000BD80	SysTick_Handler_EXIT	7.651180 ms	CORE_CM7	
	31.0	0800*08D4	0000B580	{	8.161288 ms	CORE_CM7	
				SysTick_Handler			
	31.1	0800*08D8	FF3CF7FF	InterruptRoutine();	8.161295 ms	CORE_CM7	
	31.2	0800*0754	0000B480	{	8.161299 ms	CORE_CM7	
				InterruptRoutine			
	31.3	0800*0758	00004B0E	if ((iWait++) > iWAITVALUE)	8.161306 ms	CORE_CM7	
	31.4	0000*0000	00000001	{ETM4} Event	8.161348 ms	CORE_CM7	
	31.5	0800*077E	00004B08	interfcoun++;	8.161352 ms	CORE_CM7	
	31.6	0800*078A	0000BF00	}	8.161390 ms	CORE_CM7	
	31.7	0800*0792	00004770	InterruptRoutine_EXIT	8.161409 ms	CORE_CM7	
	31.8	0800*08DC	0000BF00	}	8.161416 ms	CORE_CM7	
	31.9	0800*08DE	0000BD80	SysTick_Handler_EXIT	8.162412 ms	CORE_CM7	
	35.0	2000*0016	0000698C	interfcoun	8.163408 ms	CORE_CM7	
				{ITM} Data Write			
	35.1	0800*08D4	0000B580	{	8.163536 ms	CORE_CM7	
				SysTick_Handler			
	35.2	0800*08D8	FF3CF7FF	InterruptRoutine();	8.163543 ms	CORE_CM7	
	35.3	0800*0754	0000B480	{	8.163546 ms	CORE_CM7	
				InterruptRoutine			
	35.4	0800*0758	00004B0E	if ((iWait++) > iWAITVALUE)	8.163553 ms	CORE_CM7	
	35.5	0000*0000	00000001	{ETM4} Event	8.163592 ms	CORE_CM7	
	35.6	0800*077E	00004B08	interfcoun++;	8.256279 ms	CORE_CM7	
	35.7	0800*078A	0000BF00	}	8.812402 ms	CORE_CM7	
	35.8	0800*0792	00004770	InterruptRoutine_EXIT	9.090464 ms	CORE_CM7	

3. Productivity tips

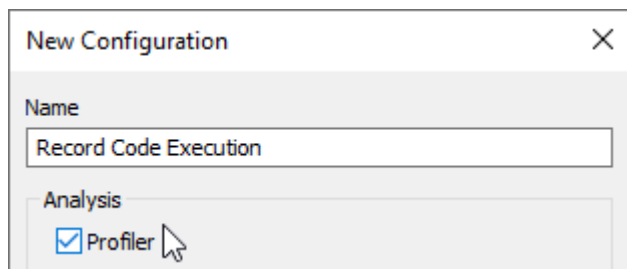
1.3 Default Trigger

Don't apply changes to the *Default Trigger* because it is always a working reference that you should keep.

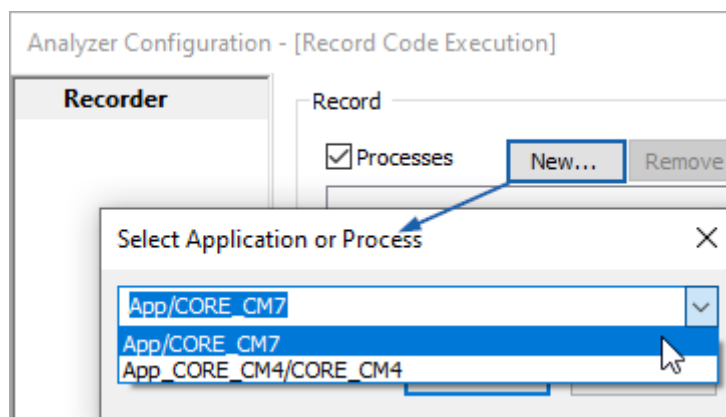
1.4 Configure Profiler

To configure Profiler additional settings are required.

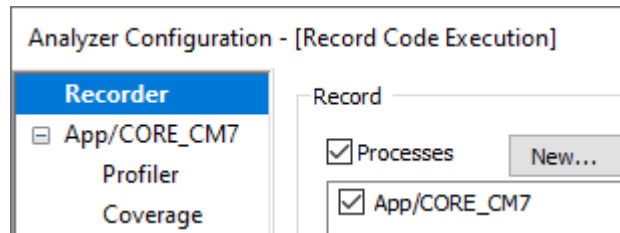
1. Make sure *Profiler* option is checked in the *New Configuration* dialog.



2. Configure Profiler in the *Analyzer Configuration* dialog.
 - a. Press *Analyzer Configuration* button .
 - b. Make sure *Processes* are checked.
 - c. Press *New* and select *App/CORE_M7*. (In this example we will only record the M7 core.)



- d. Press OK. *App/CORE_M7* page along appears in the dialog. Profiler sub-page allows you to configure Code, Data, OS object etc. Refer to [Profiler Configuration](#) in winIDEA Help for more information.

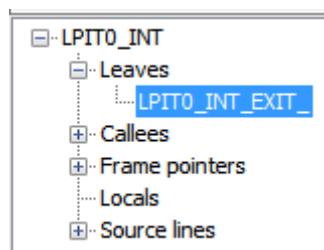


1.5 Search for variables via the Symbol Browser

Use the  button to open the *Symbol Browser* dialog and select a variable.

1.6 Discover the end of the selected function via the Symbol Browser

To discover the end of the selected function, double-click on the selected function in the Symbol Browser and use the *Leaves* or simply add manually `_EXIT_` onto end of desired function text.



1.7 Available DWT Comparators

Four DWT comparators are available on the ETMv4. You can use them in pairs e.g. for recording two different ranges of program execution.

First pair can be used for recording one program execution range while the other pair can be used to record e.g. another program execution range or 1-2 variables as an alternative.

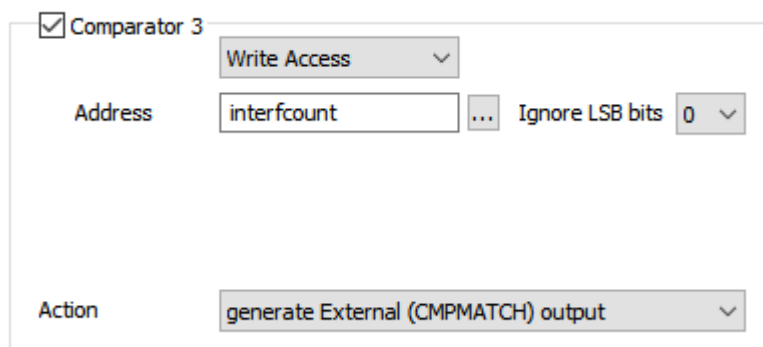
Note: ITM in the *ITM* page must be checked too for the DWT to work, regardless if ITM is used for its primary use or not.

1.8 Continuously Record Code Execution “from – until”

This is a bit more complicated example for code execution continuous recording of a range “from – until”. The goal is to trace a variable only within a certain part of the code, which writes to it.

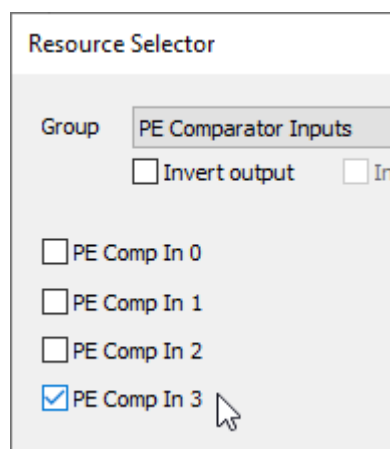
ETM4 is used as a *notifier* every time the variable is written.

1. Configure additional DWT Comparator.
 - a. Open the *DWT CORE_CM7* page and enable Comparator 3.
 - b. Select *Write Access* from the drop-down.
 - c. Add the address e.g., *interfcount*.
 - d. Select option *generate External (CMPMATCH) output* from the *Action* drop-down.



The screenshot shows the configuration for Comparator 3. It is checked and labeled 'Comparator 3'. The 'Write Access' dropdown is set to 'Write Access'. The 'Address' field contains 'interfcount'. There is an 'Ignore LSB bits' field set to '0'. The 'Action' dropdown is set to 'generate External (CMPMATCH) output'.

2. Set *Resource Selector* as internal trigger event.
 - a. Open the *ETM CORE_CM7* page.
 - b. Press **3** in the *Resource Selectors* section.
 - c. Select *PE Comparator Inputs* from the *Group* drop-down.
 - d. Check *PE Comp In 3*.



The screenshot shows the 'Resource Selector' configuration. The 'Group' dropdown is set to 'PE Comparator Inputs'. There are checkboxes for 'Invert output' and 'In'. Below these are checkboxes for 'PE Comp In 0', 'PE Comp In 1', 'PE Comp In 2', and 'PE Comp In 3'. The 'PE Comp In 3' checkbox is checked, and a mouse cursor is pointing at it.

- e. Press OK. *Resource Selector* is now set.

Note that **2** was set in the [step 3](#).

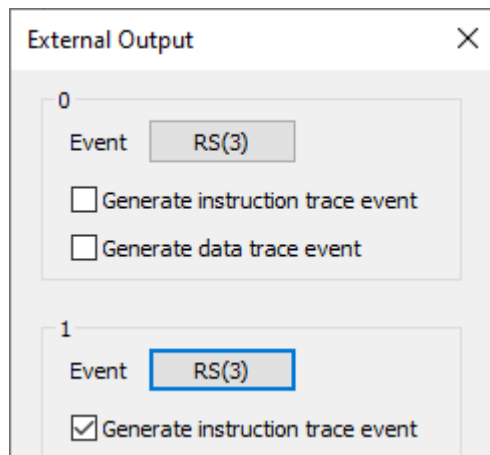
Resource Selectors	
0	FALSE
1	TRUE
2	PE(0)
3	PE(3)

3. Set *External Output*.
 - a. On the *ETM CORE_CM7* page press *External Output*.
 - b. Make sure both options are checked:
 - i. *Generate instruction trace event*
 - ii. *Generate data trace event*
 - c. Press *RS(2)* in the section 0.
 - d. Select 3: *PE(3)* from the *Resource* drop-down.

- e. Press OK. *External Output* section is now set.

External Outputs
0: I, D, RS(3)

Note that if you would use the setup below instead (where *section 1* is configured too) trace will be recorded correctly, but it won't display the results automatically if the trigger is hit.



Trace remains in the WAITING state **WAITING** and you must manually stop  the recording to display the results.

4. Technical support

1.9 More resources

[Online Help](#) ►

winIDEA and testIDEA help

[Knowledge Base](#) ►

Tips & tricks categorized by
issue type and architecture

[Tutorials](#) ►

From beginner to expert

[Technical Notes](#) ►

How-Tos for winIDEA
functionalities with scripts

[Application Notes](#) ►

How-to notes on advanced
use cases

[Webinars](#) ►

Technical webinars about
iSYSTEM tools with use cases

1.10 Contact

Please visit <https://www.isystem.com/contact.html> for contact details.

iSYSTEM has made every effort to ensure the accuracy and reliability of the information provided in this document at the time of publishing. Whilst iSYSTEM reserves the right to make changes to its products and/or the specifications detailed herein, it does not make any representations or commitments to update this document.

© iSYSTEM. All rights reserved.